Optimization Methods
Starting Task - PFSP with EA

Author Name

March 24, 2026

Contents

1 Introduction and Problem Definition **4**

1.1 Motivation and Context 4

1.2 Problem Statement 4

2 Experimental Setup and Benchmark Instances **6**

2.1 Benchmark Instances 6

2.2 Hardware and Software Environment 6

2.3 Evaluation Protocol 7

2.3.1 Stopping Criterion and Computational Budget 7

2.3.2 Budget Calibration Procedure 8

2.3.3 Seeds and Samples 8

2.4 Reported Metrics 9

2.5 Required Aggregation and Plot Statistics 9

3 Compared Optimization Methods **10**

3.1 Random Search 10

3.2 Greedy Heuristic 10

3.3 Evolutionary Algorithm (EA) 11

3.4 Simulated Annealing (SA) 11

4 Baseline Results **13**

4.1 Baseline Experiment Design 13

4.2 Baseline Results by Instance 13

5 Evolutionary Algorithm Design Details **14**

5.1 Encoding and Initialization 14

5.2 Fitness Evaluation 14

5.3 Selection, Crossover, Mutation, Replacement 14

5.4 Elitism and Termination 14

6 EA Parameter Sensitivity Analysis **15**

6.1 Single-Factor Screening Design 15

6.2 Mutation Probability 15
6.3 Crossover Probability 15
6.4 Tournament Size 15
6.5 Population Size 15
6.6 Number of Generations 15

7 Operator Role and Ablation Analysis 16
7.1 Mutation Role (on/off, low/high extremes) 16
7.2 Crossover Role (on/off, low/high extremes) 16
7.3 Elitism Role (on/off) 16

8 Joint Parameter Interaction Analysis 17
8.1 Selection Pressure x Mutation Probability 17
8.2 Selection Pressure x Crossover Probability 17
8.3 Population Size x Generations 17
8.4 Elitism x Tournament Size 17

9 Final EA Configuration 18
9.1 Selection Criteria 18
9.2 Chosen Configuration 18

10 Final Comparative Results on 6 Instances 19
10.1 Main Comparison Tables 19
10.2 Difference-from-Best Table 19

11 Convergence Analysis 20
11.1 Convergence Plot Protocol 20
11.2 Required Plots 20
11.3 Easy, Medium, and Hard Instance Behavior 20

12 Discussion and Answers to Auxiliary Questions 21
12.1 Tournament Size Effects 21
12.2 Mutation and Crossover Effects 21
12.3 Elitism, Population, and Offspring Effects 21

13 Conclusions 22
13.1 Key Findings 22
13.2 Limitations 22
13.3 Future Work 22

A Reproducibility Checklist **23**
A.1 Configuration Summary 23
A.2 Artifacts 23

Chapter 1

Introduction and Problem Definition

1.1 Motivation and Context

Permutation Flow Shop Scheduling Problem (PFSP) is an extensively studied classical combinatorial optimization problem. It appears in many real-world settings, such as manufacturing, production lines, logistics, and computer systems.

Exact methods become impractical for larger PFSP instances because the search space grows factorially with the number of jobs. For this reason, heuristic and metaheuristic methods are commonly used in practice. In this project, the main focus is on an evolutionary algorithm (EA), with comparisons to alternative methods such as random search, greedy algorithm and other metaheuristics.

1.2 Problem Statement

In a standard PFSP setting:

- processing times are known and fixed,
- each machine processes at most one job at a time,
- each job is processed by at most one machine at a time,
- no preemption is allowed (a job must be completed on a machine before it can move to the next),
- no passing is allowed (the order of jobs is the same on all machines).

In PFSP, all machines process jobs in one common permutation, so a solution is a permutation of jobs. The objective is to find a permutation that minimizes a given performance measure, such as Total Flow Time (TFT $\sum C_j$) or Makespan ($C_{\max}$).

Jobs and Machines. Jobs are indexed by $j \in \{1, \ldots, J\}$ and machines by $m \in \{1, \ldots, M\}$. Let $p(j, m) \geq 0$ denote the processing time of job j on machine m. All jobs follow the same machine order $1, 2, \ldots, M$.

Solution (Job Sequence). We seek a job sequence $x = \langle x_1, \ldots, x_J \rangle$, which is a permutation of $\{1, \ldots, J\}$ specifying the order in which jobs are processed on every machine.

Completion times For a given sequence x, let $c(x_j, m)$ be the completion time of job x_j on machine m such that:

$$c(x_j, m) = \begin{cases} p(x_j, m), & j = 1 \wedge m = 1, \\ c(x_j, m-1) + p(x_j, m), & j = 1 \wedge m > 1, \\ c(x_{j-1}, m) + p(x_j, m), & j > 1 \wedge m = 1, \\ \max\{c(x_{j-1}, m),\ c(x_j, m-1)\} + p(x_j, m), & j > 1 \wedge m > 1\ . \end{cases} \tag{1.1}$$

Objective (Total Flow Time). The most common objective in PFSP studies is to minimize the Makespan $C_{\max}$, which is the completion time of the last job on the last machine. This objective is often used in production scheduling to minimize the total time required to complete all jobs.

However, in this task, the objective is to minimize Total Flow Time (TFT), also called Total Completion Time or Sum of Completion Times. This metric reflects the total time that all jobs spend in the system, which is relevant in contexts where the average completion time of jobs is more important.

The Total Flow Time for a given sequence x can be expressed as:

$$\text{TFT} = f(x) = \sum_{j=1}^{J} c(x_j, M).$$

Let $C_j := c(x_j, M)$ denote the completion time of the jth job in the sequence on the last machine M. Then TFT $= \sum_{j=1}^{J} C_j$ The optimization problem can then be formally stated as:

$$\min_{x \in \mathcal{S}_J} f(x) = \min \sum_{j=1}^{J} C_j, \tag{1.2}$$

where $\mathcal{S}_J$ denotes all schedules (permutations) of jobs $\{1, \ldots, J\}$.[1].

[1]For $M \geq 2$ the problem is NP-hard. Lower values generally indicate shorter average completion time of jobs in the system.

Chapter 2

Experimental Setup and Benchmark Instances

2.1 Benchmark Instances

We run all experiments on the six default PFSP benchmark instances provided by the repository at github.com/chneau/go-taillard (Taillard flow-shop instances). These six instances are fixed across all experiments to ensure fair, apples-to-apples comparisons. We use the processing-time matrices exactly as distributed in the repository. We will also report the solutions versus the best-known objective values provided for these instances.

For reporting, we refer to each instance by its repository filename and include its size (n, m) in the result tables.

Table 2.1: Benchmark instances used in the experiments

Description	Filename	n	m
Small instance 1	`tai_20_5_0`	20	5
Small instance 2	`tai_20_10_0`	20	10
Small instance 3	`tai_20_20_0`	20	20
Medium instance 1	`tai_100_10_0`	100	10
Medium instance 2	`tai_100_20_0`	100	20
Large instance 1	`tai_500_20_0`	500	20

2.2 Hardware and Software Environment

Note: Replace the placeholders in Table 2.2 with your actual setup before finalizing the report.

Table 2.2: Hardware and software environment for reproducibility

Item	Value
CPU	AMD Ryzen 7 9800X3D
Cores/Threads	8 / 16
RAM	64 GB
Operating System	Windows 11
Python Environment	Python 3.13.12; uv-managed virtual environment
Python Libraries	matplotlib $\geq$ 3.10.8; notebook $<$ 7; pandas $\geq$ 3.0.1; scipy $\geq$ 1.17.1
C# / .NET SDK	Microsoft.NET.Sdk; TargetFramework: net10.0
NuGet Packages	BenchmarkDotNet 0.15.2; BenchmarkDotNetDiagnosers; Newtonsoft.Json 13.
Parallelism	Only parallel experiments, no algorithm-level parallelism

2.3 Evaluation Protocol

2.3.1 Stopping Criterion and Computational Budget

A common stopping criterion was defined using the **maximum number of objective function evaluations**. Let B denote the computational budget assigned to a single run on a given instance. For the evolutionary algorithm, this budget is derived from its main control parameters as

$$B = \text{pop_size} \times \text{gen},$$

with the initial population evaluations for EA not included for simplicity, counted as an algorithm set-up cost instead. The same value of B is then imposed on the remaining methods, so that all are compared under an equal limit on the number of candidate solutions evaluated.

An exception is made for the greedy algorithm, which always returns exactly one solution.

Limitations - this budget definition provides a method, implementation and hardware independent comparison criterion. However, it is not free from limitations. The actual computational cost of a single evaluation may differ between methods, especially when one method uses incremental evaluation, fitness caching, or partial recomputation. In addition, algorithms may spend a non-negligible amount of time in operations not directly counted as evaluations, such as neighborhood generation. Therefore, the evaluation budget should be interpreted as a fair *algorithmic* budget rather than a practical cost.

Alternatives include a wall-clock time budget, quality-based stopping, efficiency-based criteria, parallellism-aware budgets and others. They may, however, introduce unwanted variables such as hardware and software dependence, algorithm-specific conditions, or others such as implementation quality or stochastic variance.

2.3.2 Budget Calibration Procedure

A **single fixed computational budget** B simplifies the experimental design and ensures that all algorithms operate under identical limits. However, this approach introduces an inherent trade-off: instances of different sizes and difficulty levels may require different amounts of computational effort to reach comparable solution quality. In particular, larger and more complex instances may benefit from higher budgets. Consequently, this choice may disadvantage larger and over-allocate effort to smaller instances, where improvements beyond a certain point are marginal.

To address this issue in a controlled manner, a **budget calibration procedure** was conducted prior to the main experiments. Preliminary runs were performed using representative algorithms—specifically, the evolutionary algorithm (EA) and simulated annealing (SA) with default parameter settings. These pilot experiments were executed across selected instances of varying sizes, and several candidate budget values were tested. The evaluation focused on three criteria: (i) achieved solution quality, (ii) runtime, and (iii) convergence behavior (efficiency of improvement over time).

Based on the results, the budget was chosen by arbitrarily selecting a value that, based on observed trends, was expected to provide a reasonable baseline for all instances.

2.3.3 Seeds and Samples

Seeds To ensure full reproducibility of the experiments, all stochastic algorithms were executed using explicitly defined random seeds.

For each combination of algorithm, problem instance, and parameter configuration, multiple independent runs were performed. Each run was assigned a distinct seed computed according to the formula:

$$\text{seed}_i = \text{base_seed} \oplus (i \cdot K), \quad \forall, i \in 0, 1, \ldots, n-1, ; K \in 2\mathbb{Z} + 1$$

where i is the run index, K is a large odd integer constant, and $\oplus$ denotes the bitwise XOR operation.

This approach has several advantages. First, it guarantees that the same experimental setup always produces identical results. Second, by combining a base seed with an index, it avoids using consecutive seeds. The use of a large odd multiplier combined with the XOR operation reduces the risk of output correlations. This balances reproducibility and

independence of runs.

Sample size In the screening experiments, $\mathbf{n = 10}$ runs were performed for each configuration. In the final comparative experiments, a sample of $\mathbf{n = 30}$ was used, which is commonly accepted basis for comparison in literature and should provide a distribution of the mean that approaches normal.

2.4 Reported Metrics

2.5 Required Aggregation and Plot Statistics

Chapter 3

Compared Optimization Methods

3.1 Random Search

Random Search is used as a stochastic baseline. The method repeatedly generates random job permutations, evaluates each candidate using the PFSP objective function, and keeps the best solution found so far.

In this project, the algorithm has two execution modes: sequential and parallel. Both modes share the same core logic and differ only in how random samples are distributed across workers. The parallel mode was not used in the final experiments.

Stopping is controlled either by a fixed evaluation budget or by a time limit. Each run reports the best solution value, total number of evaluations, and elapsed runtime. In this study, only the run count budget was used, as mentioned in the evaluation protocol.

3.2 Greedy Heuristic

Two deterministic constructive baselines are used in this project:

Greedy The Greedy algorithm builds a schedule without insertion search. It computes a priority value for each job as the sum of processing times across all machines, sorts jobs by this priority (ascending), and returns the resulting permutation directly. This gives a very fast baseline with exactly one constructed solution.

SPT (NEH-like iterative greedy). The SPT algorithm uses the same initial priority order, but applies an iterative insertion strategy. Starting from a partial sequence, each next job is inserted into the position that yields the best objective value for the current prefix. This is an NEH-like constructive approach: it remains deterministic and single-pass, but performs local positional choice at each insertion step.

Both methods return one final permutation (no multi-start search), making them suitable as low-cost references against stochastic metaheuristics.

3.3 Evolutionary Algorithm (EA)

The Evolutionary Algorithm is implemented as a population-based stochastic search. It starts from a randomly generated initial population, evaluates all individuals, and then iteratively creates new generations through selection, crossover, mutation, and full replacement of the population.

At each generation, two parents are selected (tournament selection), offspring are produced with a crossover probability, each child is optionally mutated with a mutation probability, and offspring are evaluated and inserted into the next population.

Default configuration used when not otherwise specified:

- Population size: 100
- Generations: 100
- Crossover rate: 0.7
- Mutation rate: 0.1
- Tournament size: 5
- Selection method: Tournament
- Crossover operator: OX (Order Crossover)
- Mutation operator: Swap

The projects implements two Mutation operators: Swap and TODOMutation, and two Crossover operators: OX and TODOX. For selection, only Tournament is implemented, with design allowing for future extension.

3.4 Simulated Annealing (SA)

Simulated Annealing is implemented as a neighborhood-based local search. The algorithm starts from a permutation, generates one neighbor at each iteration, evaluates it, applies an acceptance rule, and then updates the temperature according to a cooling schedule.

In the experiments, SA parameters are kept fixed and are not tuned. They are treated as arbitrary default settings used only to provide a stable baseline for comparison. Default configuration used in the experiment runner:

- Iterations: 10,000
- Initial temperature: $T_0 = 100.0$
- Cooling rate: $\alpha = 0.995$ (not used in linear schedule)
- Minimum temperature: $T_{\min} = 10^{-4}$
- Neighborhood operator: Swap
- Acceptance function: Probabilistic (Metropolis-type)
- Cooling schedule: Linear

For the default linear cooling schedule (as implemented), temperature is updated as

$$T_{k+1} = 1 - \frac{k}{K},$$

where k is the current iteration index (starting from 0 in this form) and K is the maximum number of iterations. In the current implementation, the linear schedule does not use α directly.

For the default probabilistic acceptance function, let $f(x)$ be the current solution cost and $f(x')$ the candidate cost. Acceptance is defined by

$$P(\text{accept } x' \mid x, T) = \begin{cases} 1, & f(x') \leq f(x), \\ \exp\left(-\dfrac{f(x') - f(x)}{T}\right), & f(x') > f(x). \end{cases}$$

Thus, improving moves are always accepted, while worsening moves are accepted with a probability that decreases as temperature decreases.

Chapter 4

Baseline Results

4.1 Baseline Experiment Design

4.2 Baseline Results by Instance

Chapter 5

Evolutionary Algorithm Design Details

5.1 Encoding and Initialization

5.2 Fitness Evaluation

5.3 Selection, Crossover, Mutation, Replacement

5.4 Elitism and Termination

Chapter 6

EA Parameter Sensitivity Analysis

6.1 Single-Factor Screening Design

6.2 Mutation Probability

6.3 Crossover Probability

6.4 Tournament Size

6.5 Population Size

6.6 Number of Generations

Chapter 7

Operator Role and Ablation Analysis

7.1 Mutation Role (on/off, low/high extremes)

7.2 Crossover Role (on/off, low/high extremes)

7.3 Elitism Role (on/off)

Chapter 8

Joint Parameter Interaction Analysis

8.1 Selection Pressure x Mutation Probability

8.2 Selection Pressure x Crossover Probability

8.3 Population Size x Generations

8.4 Elitism x Tournament Size

Chapter 9

Final EA Configuration

9.1 Selection Criteria

9.2 Chosen Configuration

Chapter 10

Final Comparative Results on 6 Instances

10.1 Main Comparison Tables

10.2 Difference-from-Best Table

Table 10.1: Template: Difference from Best by Instance

Instance	Method	f_{method}	Δ	$\Delta\%$

Chapter 11

Convergence Analysis

11.1 Convergence Plot Protocol

11.2 Required Plots

11.3 Easy, Medium, and Hard Instance Behavior

Chapter 12

Discussion and Answers to Auxiliary Questions

12.1 Tournament Size Effects

12.2 Mutation and Crossover Effects

12.3 Elitism, Population, and Offspring Effects

Chapter 13

Conclusions

13.1 Key Findings

13.2 Limitations

13.3 Future Work

Appendix A

Reproducibility Checklist

A.1 Configuration Summary

A.2 Artifacts